NoSQL Database Design_LAB SHEET - 5

1. Install and configure Riak KV on a Linux system. Create a bucket named `students` and perform basic **key-value operations** such as storing, retrieving, updating, and deleting student records.

2. Using **Riak KV**, create a `user_sessions` bucket to store **user session information**. Store session ID, user ID, login time, and session status, and retrieve session information using its key.

3. Design a Riak KV application for storing **user profiles and preferences**. Store details such as name, email, city, language, theme, and notification preferences. Demonstrate **PUT and GET operations**.

4. Implement a **shopping cart application using Riak KV**. Store a shopping cart for each user containing product IDs, quantities, and prices. Demonstrate adding, updating, retrieving, and removing products from the cart.

5. Demonstrate the **structure of data in a key-value store** using Riak. Store different types of values such as strings, JSON documents, and lists. Retrieve the stored values and explain the role of **bucket, key, and value**.

6. Demonstrate **consistency in Riak KV** by configuring suitable read and write quorum values. Perform read/write operations and explain how Riak handles consistency in a distributed environment.

7. Design a Riak-based application that demonstrates **scaling**. Create multiple buckets and insert a large number of key-value records. Explain how Riak distributes data across nodes and supports horizontal scaling.

8. Implement a **multi-operation transaction-like scenario** using Riak KV for a shopping application. Perform multiple related operations such as adding an item to a cart, updating its quantity, and updating the cart total. Discuss the limitations of transactions in a key-value database.

9. Using Riak, implement **query-by-data** functionality for user records. Store JSON documents containing user information and demonstrate how application-level logic can be used to locate users based on attributes such as city, age, or preferences.

10. Implement **set operations** in a Riak-based application. Create sets containing user interests or product categories and demonstrate operations such as **adding elements, removing elements, checking membership, and finding common elements**.

11. Install and configure **Neo4j**. Create a graph containing `Student`, `Course`, and `Department` nodes with appropriate relationships. Insert sample data and visualize the resulting graph.

12. Using **Neo4j Cypher**, create a social-network graph containing `Person` nodes and relationships such as `FRIENDS_WITH`, `FOLLOWS`, and `WORKS_AT`. Write queries to find a person's friends and followers.

13. Design a **shopping/e-commerce graph using Neo4j** containing `Customer`, `Product`, `Category`, and `Order` nodes. Create appropriate relationships and write Cypher queries to find products purchased by a particular customer.

14. Demonstrate **transactions, consistency, and availability in Neo4j**. Perform multiple `CREATE`, `MATCH`, `SET`, and `DELETE` operations inside a transaction and explain how transaction management maintains consistent graph data.

15. Develop a small **Graph Database application using Neo4j and a programming language such as Python, Java, or JavaScript**. Implement CRUD operations and Cypher queries. Demonstrate how graph databases support **relationship-based queries, scaling, availability, and suitable use cases** such as social networks, recommendation systems, fraud detection, and knowledge graphs.